



دانشکده علوم ریاضی
دانشگاه فردوسی مشهد

پروژه دوم درس طراحی الگوریتم

فاز سوم: یافتن بهینه‌ترین مجموعه کابل‌های پشتیبان

استاد درس: دکتر رضا قنبری

اعضای گروه:

سارا مسگر مقدم، اسما موحدی تبار

فهرست مطالب

۱	مقدمه	۲
۲	عدم بهینگی راهبرد حریصانه	۲
۱.۲	مثال نقض	۲
۲.۲	نتیجه‌گیری از مثال نقض	۳
۳	الگوریتم شاخه و کران (Branch and Bound)	۳
۱.۳	ساختار درخت جستجو و وضعیت گره‌ها	۳
۲.۳	کران پایین (Lower Bound)	۴
۳.۳	کران بالا (Upper Bound)	۵
۴.۳	شرایط هرس (Pruning)	۵
۴	مقایسه خروجی فاز دوم و فاز سوم	۶
۵	نتیجه‌گیری	۶

۱ مقدمه

در فاز دوم پروژه، یک الگوریتم حریصانه برای انتخاب کابل‌های پشتیبان ارائه شد تا شبکه در برابر قطع شدن هر یک از یال‌های درخت پوشای کمینه مقاوم شود. در این گزارش، ابتدا تعریف دقیق معیار حریصانه‌ی پیاده‌سازی شده در فاز دوم را بیان کرده، سپس با ارائه‌ی یک مثال نقض، نشان می‌دهیم که این راهبرد لزوماً به جواب بهینه منتهی نمی‌شود. در ادامه، یک الگوریتم دقیق مبتنی بر Branch and Bound برای یافتن جواب بهینه معرفی شده و ساختار درخت جستجو، تابع کران پایین و شرایط هرس آن به‌طور کامل تشریح می‌شود.

۲ عدم بهینگی راهبرد حریصانه

اگرچه الگوریتم حریصانه‌ی فوق سریع است، اما تصمیم‌گیری‌های کاملاً محلی و مستقل از آینده، باعث می‌شود که نتواند در همه‌ی موارد جواب بهینه را تولید کند. در ادامه، دو مورد خواسته‌شده در صورت پروژه را با یک مثال نقض واحد بررسی می‌کنیم.

۱.۲ مثال نقض

فرض کنید درخت پوشای کمینه دارای سه یال e_1, e_2, e_3 است (یک مسیر با ۴ رأس). کابل‌های پشتیبان به‌صورت زیر هستند:

کابل	هزینه	یال‌های پوشش داده‌شده
A	۵	$\{e_1, e_2\}$
B	۵	$\{e_3\}$
C	۶	$\{e_1, e_2, e_3\}$

الگوریتم حریصانه (یال‌محور):

۱. برای یال e_1 : ارزان‌ترین کابلی که e_1 را پوشش دهد، کابل A با هزینه‌ی ۵ است -> انتخاب A .
۲. برای یال e_2 : کابل B با هزینه‌ی ۵ را پوشش می‌دهد. -> انتخاب B .
۳. برای یال e_3 : کابلی که e_3 را پوشش دهد و هنوز انتخاب نشده باشد، کابل C با هزینه‌ی ۶ است -> انتخاب C .

هزینه کل حریصانه: $5 + 5 + 6 = 16$

جواب بهینه: انتخاب تنها کابل C با هزینه‌ی ۶ که هر سه یال را پوشش می‌دهد.

جواب بهینه	راهبرد حریصانه	
C	A, B, C	کابل‌های انتخاب‌شده
۶	۱۶	هزینه کل

۲.۲ نتیجه‌گیری از مثال نقض

مثال فوق به‌وضوح نشان می‌دهد که:

۱. الگوریتم حریصانه، کم‌هزینه‌ترین زیرگراف دو-یال-همبند ممکن را تولید نمی‌کند (چون زیرگراف حاصل از A و C و B با هزینه‌ی ۱۶ در حالی که زیرگراف C با هزینه‌ی ۶ وجود دارد).

۲. الگوریتم حریصانه، کم‌هزینه‌ترین روش برای تبدیل درخت پوشای کمینه به یک زیرگراف دو-یال-همبند را تولید نمی‌کند (چون تبدیل با A و C و B هزینه‌ی ۱۶ دارد، در حالی که تبدیل با C هزینه‌ی ۶ دارد).

بنابراین، هیچ تضمینی برای بهینگی الگوریتم حریصانه وجود ندارد و برای یافتن جواب بهینه باید از روش‌های دقیق‌تری مانند Branch and Bound استفاده کرد.

۳ الگوریتم شاخه و کران (Branch and Bound)

برای یافتن جواب بهینه، از روش شاخه و کران استفاده شده است. این روش تمام فضای حالت را به‌صورت سیستماتیک جستجو می‌کند، اما با استفاده از کران‌های پایین و بالا، شاخه‌های غیرامیدوارکننده را هرس می‌کند.

۱.۳ ساختار درخت جستجو و وضعیت گره‌ها

هر گره در درخت جستجو، یک وضعیت (State) را نشان می‌دهد که شامل اطلاعات زیر است:

- index: اندیس کابل بعدی که باید درباره‌ی آن تصمیم‌گیری شود.

- cost: مجموع هزینه کابل‌های انتخاب‌شده تا این گره.

• covered: یک آرایه بولی به طول $n - 1$ که مشخص می‌کند کدام یال‌های MST پوشش داده شده‌اند.

• selected: لیست کابل‌های انتخاب‌شده در این مسیر.

ریشه درخت: در ابتدا هیچ کابلی انتخاب نشده، هزینه صفر، همه‌ی یال‌ها پوشش نشده و اندیس صفر است.

نحوه انشعاب: (Branching) در هر گره، برای کابل فعلی (`backups.get(index)`) دو شاخه ایجاد می‌شود:

۱. شاخه انتخاب: (Take) کابل فعلی انتخاب می‌شود. هزینه به‌روز می‌شود، یال‌های پوشش داده‌شده علامت‌گذاری می‌شوند و کابل به لیست انتخاب‌شده‌ها اضافه می‌شود. سپس `index` یک واحد افزایش می‌یابد.

۲. شاخه رد: (Skip) کابل فعلی رد می‌شود. تنها `index` یک واحد افزایش می‌یابد.

این فرآیند به صورت بازگشتی (عمق‌اول) ادامه می‌یابد تا زمانی که:

• همه‌ی یال‌ها پوشش داده شوند (جواب کامل).

• یا شاخه هرس شود.

• یا همه‌ی کابل‌ها بررسی شوند.

۲.۳ کران پایین (Lower Bound)

طراحی یک کران پایین معتبر، مهم‌ترین بخش الگوریتم شاخه و حد است. در ابتدا، کران پایینی مبتنی بر جمع کمترین هزینه‌های تکی برای هر یال پوشش نشده در نظر گرفته شد:

$$LB_1(state) = state.cost + \sum_{i: uncovered} \min_{b \text{ covers } i} cost(b)$$

اما پس از تحلیل دقیق، مشخص شد که این کران معتبر نیست، زیرا ممکن است یک کابل با هزینه‌ی کم، چندین یال را هم‌زمان پوشش دهد و هزینه‌ی واقعی جواب بهینه بسیار کمتر از این جمع باشد. در نتیجه، این کران می‌توانست جواب بهینه را هرس کند. بنابراین این روش کنار گذاشته شد و کران پایین معتبر زیر جایگزین گردید.

تعریف کران پایین نهایی:

$$LB(state) = state.cost + \min_{i \geq state.index} \left\{ cost(backup_i) \mid backup_i \text{ edge uncovered one least at covers} \right\}$$

به عبارت دیگر، از بین کابل‌های باقی‌مانده، ارزان‌ترین کابلی که حداقل یک یال پوشش‌نشده را پوشش می‌دهد، انتخاب شده و هزینه‌ی آن به هزینه‌ی فعلی اضافه می‌شود.

اثبات اعتبار: فرض کنید S یک جواب کامل دلخواه از وضعیت فعلی باشد. در S حداقل یک کابل وجود دارد که یک یال پوشش‌نشده را پوشش می‌دهد. هزینه‌ی آن کابل نمی‌تواند از $\min_{i \geq \text{state.index}} \text{cost}(\text{backup}_i)$ کمتر باشد، زیرا این مقدار کمترین هزینه‌ی ممکن در میان تمام کابل‌های باقی‌مانده است. بنابراین هزینه‌ی کل S حداقل $\text{state.cost} + \min_remaining$ است. پس $LB(\text{state}) \leq \text{cost}(S)$ برای هر جواب کامل S ، و در نتیجه $LB(\text{state})$ هرگز جواب بهینه را هرس نمی‌کند. $\square$

این کران پایین اگرچه ساده است، اما معتبر بوده و در عمل هرس قابل قبولی ارائه می‌دهد.

۳.۳ کران بالا (Upper Bound)

برای بهبود کارایی، یک جواب اولیه با استفاده از الگوریتم حریصانه (همان روش فاز دوم) ساخته می‌شود و به عنوان bestCost (بهترین جواب یافت‌شده تا کنون) در نظر گرفته می‌شود. این کار باعث می‌شود هرس‌ها زودتر انجام شوند.

۴.۳ شرایط هرس (Pruning)

در هر گره، قبل از انشعاب، سه نوع هرس اعمال می‌شود:

۱. **هرس هزینه:** اگر $\text{state.cost} \geq \text{bestCost}$ ، این شاخه نمی‌تواند جواب بهتری از جواب فعلی داشته باشد، پس هرس می‌شود.

۲. **هرس امکان‌پذیری:** اگر با کابل‌های باقی‌مانده نتوان همه‌ی یال‌ها را پوشش داد (بررسی با تابع canStillCover)، شاخه هرس می‌شود.

۳. **هرس کران پایین:** اگر $\text{lowerBound}(\text{state}) \geq \text{bestCost}$ ، حتی بهترین حالت ممکن از این گره نیز از جواب فعلی بدتر است، پس هرس می‌شود.

ترکیب این سه معیار، فضای جستجو را به شدت کاهش می‌دهد و الگوریتم را برای ورودی‌های متوسط کارآمد می‌سازد.

۴ مقایسه خروجی فاز دوم و فاز سوم

برای مثال نقض ارائه‌شده در بخش ۳، خروجی دو فاز به صورت زیر است:

فاز سوم (Branch) and (Bound)	فاز دوم (حریصانه)	
C	A, C, B	کابل‌های انتخاب‌شده
۶	۱۶	هزینه کل
بهینه	غیربهینه	بهینگی

این جدول به وضوح نشان می‌دهد که الگوریتم حریصانه (با معیار یال‌محور) جواب بهینه را از دست می‌دهد، در حالی که روش شاخه و حد با هرس‌های مناسب به جواب بهینه می‌رسد.

۵ نتیجه‌گیری

در این گزارش، ابتدا تعریف دقیق معیار حریصانه‌ی یال‌محور پیاده‌سازی شده در فاز دوم ارائه شد. سپس با ارائه‌ی یک مثال نقض ساده و قانع‌کننده، نشان داده شد که این راهبرد:

۱. الزاماً کم‌هزینه‌ترین زیرگراف دو-یال-همبند را تولید نمی‌کند.
۲. الزاماً کم‌هزینه‌ترین روش برای تبدیل درخت پوشای کمینه به یک زیرگراف دو-یال-همبند را تولید نمی‌کند.

در ادامه، الگوریتم شاخه و حد به‌عنوان یک روش دقیق معرفی شد و ساختار درخت جستجو، نحوه‌ی انشعاب و نمایش وضعیت هر گره به‌طور کامل تشریح گردید. همچنین یک کران پایین معتبر با اثبات ریاضی ارائه شد که تضمین می‌کند هیچ جواب بهینه‌ای در فرآیند هرس از بین نرود.

در مجموع، اگرچه الگوریتم شاخه و حد در بدترین حالت دارای پیچیدگی نمایی است، اما استفاده از کران پایین معتبر، کران بالا و شرایط هرس مناسب باعث می‌شود بخش قابل توجهی از فضای جستجو حذف شود و جواب بهینه در زمان قابل قبولی به دست آید. بنابراین این روش نسبت به الگوریتم حریصانه، ضمن حفظ تضمین بهینگی، عملکرد عملی مناسبی نیز برای اندازه‌های متوسط مسئله ارائه می‌دهد.